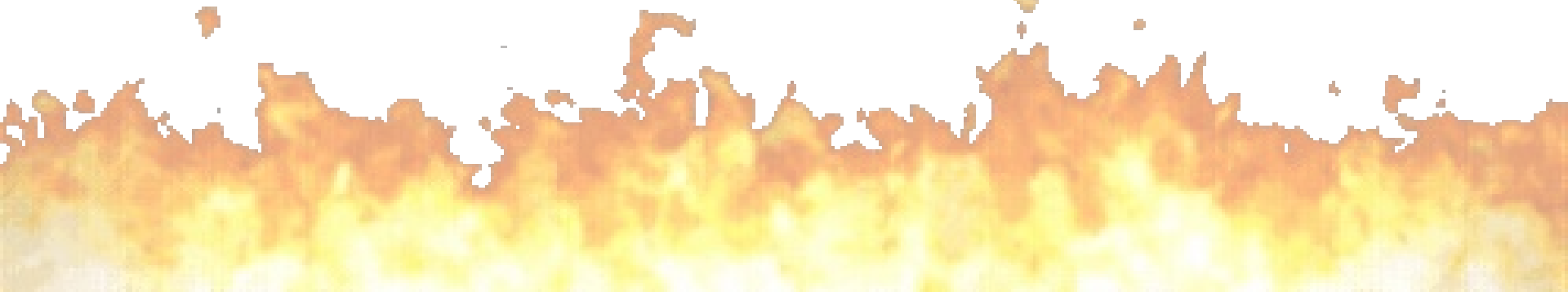


뉴런데브클래스

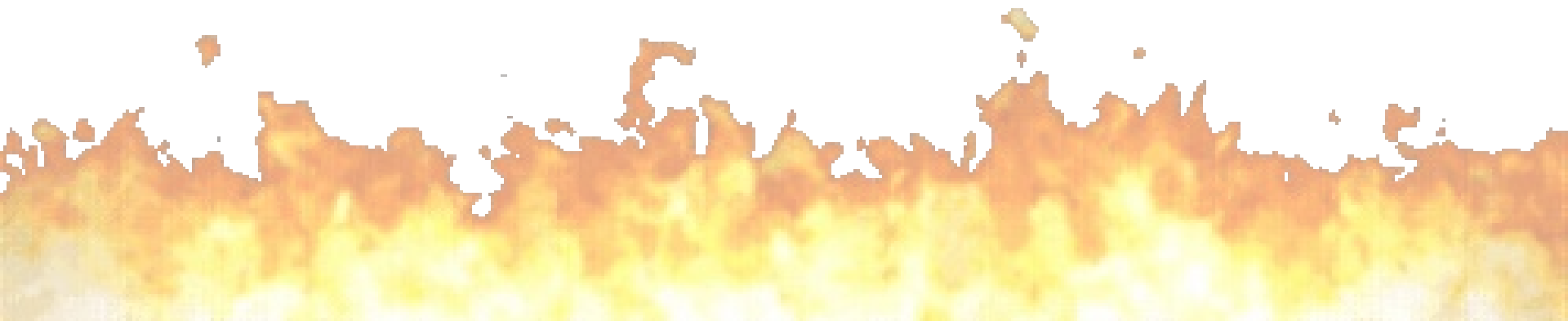
현대 AI 의 원리와 구조

#4-1 벡터디비 실습

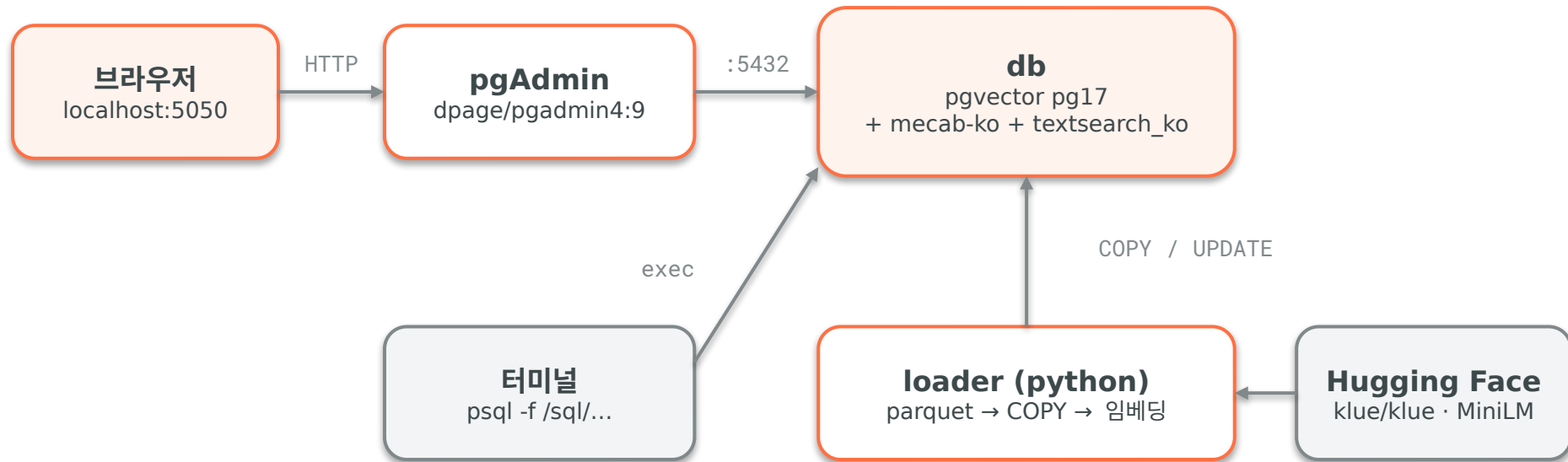


목차

- 1 실습 환경 구성
- 2 데이터 적재
- 3 형태소 분석과 tsvector
- 4 ts_rank 계열의 동작
- 5 BM25 직접 구현
- 6 종합 비교와 해석



1. 강의에서 다룬 키워드 검색을 PostgreSQL 안에서 직접 돌려 본다
 - 한국어 형태소 분석 → tsvector → ts_rank · ts_rank_cd
2. PG 에 없는 BM25 를 테이블과 SQL 만으로 구현한다
 - TF 역색인 · DF · IDF · 문서 길이 · 전역 통계
 - 트리거로 문서 추가 · 수정 · 삭제를 실시간 반영
3. 정답이 있는 한국어 데이터로 검색 품질을 숫자로 비교한다
 - KLUE-MRC 질문 500 개 → Hit@10 · MRR
 - ts_rank vs BM25 vs 벡터 vs 하이브리드
4. 준비물은 Docker Desktop 하나
 - 컴포즈 한 번으로 DB · 형태소 분석기 · pgAdmin · 로더까지



볼륨 : pgdata (DB) · pgadmin (설정) · hfcache (데이터셋 · 모델 캐시)
./sql 폴더는 db 의 /sql 과 pgAdmin 파일 브라우저에 동시에 마운트

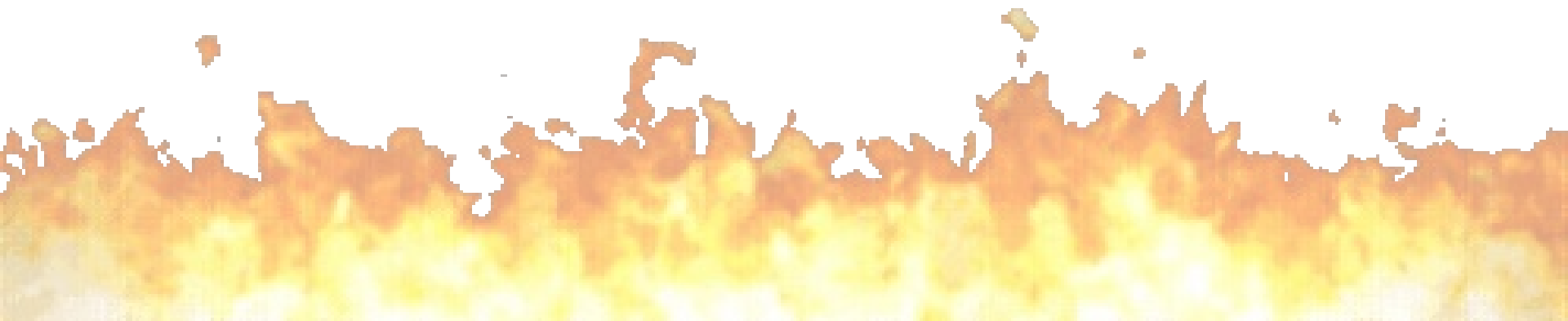
60 분 진행표

기계가 기다리는 시간은 10 분 남짓

시간	단계	내용	실측 대기 (12 코어)
00-10	① 환경 구성	compose up · Dockerfile 설명	1 분 31 초
10-13	② pgAdmin	접속 · Query Tool	-
13-16	③ 데이터 적재	loader → 텍스트 적재 완료	1 분 (임베딩은 뒤에서 3 분)
16-33	④ tsvector · ts_rank	sql/10 ~ 14	평가 2 분
33-52	⑤ BM25 구현	sql/20 ~ 24	구축 30 초
52-60	⑥ 종합 비교	sql/25	30 초

캐시 없는 최초 실행 : compose up 1:31 → 텍스트 적재 완료 2:33 → 임베딩 완료 5:23
 4 코어 노트북 · 느린 회선에서 2~3 배 걸려도 임베딩은 ④⑤ 를 하는 동안 끝난다

실습 환경 구성



docker-compose.yml

up 한 번에 DB 와 pgAdmin, 로더는 필요할 때 run

```
services:
  db:
    build: ./db # 형태소 분석기를 넣은 이미지
    ports: ["${DB_PORT:-5432}:5432"]
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./db/init:/docker-entrypoint-initdb.d:ro
      - ./sql:/sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U lab -d lab"]
  pgadmin:
    image: dpage/pgadmin4:9
    ports: ["${PGADMIN_PORT:-5050}:80"]
    depends_on: {db: {condition: service_healthy}}
  loader:
    build: ./loader
    profiles: ["tools"] # run 으로만 실행
    volumes: [hfcache:/cache]
```

- 1.healthcheck
-db 준비 후 pgAdmin-loader 기동
- 2.init 폴더
- 데이터가 빈 첫 기동에만 실행
- 3.profiles: tools
-docker compose run --rm loader
4. 포트 충돌
-DB_PORT=15432 로 실행
5. 처음부터
-docker compose down -v

왜 형태소 분석기인가

PG 기본 설정에는 한국어 사전이 없다

'고양이가 사료를 먹었습니다 . 고양이 사료 추천 '

설정	tsvector
simple	' 고양이 ':4 ' 고양이가 ':1 ' 먹었습니다 ':3 ' 사료 ':5 ' 사료를 ':2 ' 추천 ':6
korean	' 고양이 ':1,4 ' 먹 ':3 ' 사료 ':2,5 ' 추천 ':6

1.simple 은 공백으로만 자른다

- '고양이가' 와 '고양이' 가 다른 토큰 → '고양이' 로 검색하면 첫 문장을 못 찾는다

2.korean = mecab-ko 형태소 분석

- 조사 (가 · 를) · 어미 (었습니다) 를 떼고 , 용언은 어간 (먹) 으로

3. 구성 요소 3 개

-mecab-ko : 형태소 분석 엔진 (C++)

-mecab-ko-dic : 세종 말뭉치 기반 사전

-textsearch_ko : mecab 을 PG 전문검색 파서 · 사전으로 연결하는 익스텐션

Dockerfile ① 빌드 단계

pgvector 공식 이미지 위에서 세 가지를 소스 빌드

```
FROM pgvector/pgvector:pg17 AS builder
RUN apt-get install -y build-essential autoconf automake libtool \
    autotools-dev curl postgresql-server-dev-17

# 1) mecab-ko 엔진
RUN curl -fsSL "$MECAB_URL" | tar xz && cd mecab-*-ko-* \
    && cp /usr/share/misc/config.guess /usr/share/misc/config.sub . \
    && ./configure --prefix=/usr/local && make -j"$(nproc)" && make install

# 2) mecab-ko-dic 사전
RUN curl -fsSL "$MECAB_DIC_URL" | tar xz && cd mecab-ko-dic-* \
    && ./autogen.sh \
    && cp /usr/share/misc/config.guess /usr/share/misc/config.sub . \
    && ./configure --with-dicdir=/usr/local/lib/mecab/dic/mecab-ko-dic \
    && make && make install

# 3) textsearch_ko : PG 익스텐션 (PGXS)
RUN curl -fsSL "$TEXTSEARCH_KO_URL" | tar xz && cd textsearch_ko-* \
    && make USE_PGXS=1 && make USE_PGXS=1 install
```

Dockerfile ② ARM 함정

산출물만 복사하는 멀티스테이지 빌드

```
FROM pgvector/pgvector:pg17
COPY --from=builder /usr/local/lib/ /usr/local/lib/          # libmecab + 사전
COPY --from=builder /usr/local/etc/mecabrc /usr/local/etc/mecabrc
COPY --from=builder /usr/lib/postgresql/17/lib/ts_mecab_ko.so ...
COPY --from=builder /usr/share/postgresql/17/extension/textsearch_ko* ...
RUN ldconfig && echo '형태소 분석기 동작 확인' | mecab | head -3
```

처음 빌드했을 때 (Windows ARM 노트북)

```
checking build system type... Invalid configuration `aarch64-linux-gnu':
machine `aarch64' not recognized
configure: error: /bin/bash ./config.sub aarch64-linux-gnu failed
```

1. 원인 : 소스에 동봉된 config.guess / config.sub 가 2014 년판 → arm64 를 모른다
2. 해결 : autotools-dev 의 최신 파일로 덮어쓰기 → amd64 · Apple Silicon · Windows ARM 모두 빌드

초기화 스크립트와 스키마

데이터 폴더가 빈 첫 기동에만 실행된다

```
-- 01_extensions.sql
CREATE EXTENSION vector;          -- pgvector
CREATE EXTENSION textsearch_ko;   -- parser/config 'korean'
ALTER DATABASE lab SET default_text_search_config = 'korean';

-- 02_schema.sql
CREATE TABLE docs (
  id          bigserial PRIMARY KEY,
  doc_key     text UNIQUE NOT NULL,  -- 지문 md5
  title       text,
  content     text NOT NULL,
  tsv         tsvector GENERATED ALWAYS AS (
    setweight(to_tsvector('korean', coalesce(title, '')), 'A') ||
    setweight(to_tsvector('korean', content), 'D')) STORED,
  embedding   vector(384));
CREATE INDEX docs_tsv_gin ON docs USING gin (tsv);

CREATE TABLE questions (id bigserial PRIMARY KEY,
  doc_id bigint REFERENCES docs,    -- 정답 지문
  question text, answer text, embedding vector(384));
```

1. tsv 는 생성 컬럼
- INSERT · UPDATE 때 형태소 분석까지 자동
2. 강의 슬라이드의 setweight 예제
- 제목 A · 본문 D
3. GIN 인덱스
- @@ 매칭용 역색인
4. questions.doc_id
- 품질 측정의 정답

```
pgadmin:
  environment:
    PGADMIN_CONFIG_SERVER_MODE: "False"
    PGADMIN_CONFIG_MASTER_PASSWORD_REQUIRED: "False"
  entrypoint: >
    /bin/sh -c "cp /pgadmin4/pgpass.src /var/lib/pgadmin/pgpass
    && chmod 600 /var/lib/pgadmin/pgpass && exec /entrypoint.sh"
  volumes:
    - ./pgadmin/servers.json:/pgadmin4/servers.json:ro
    - ./pgadmin/pgpass:/pgadmin4/pgpass.src:ro
# servers.json : "Host": "db", "Username": "lab", "PassFile": "/var/lib/pgadmin/pgpass"
# pgpass       : db:5432:*:lab:lab
```

1. 함정 : libpq 는 권한이 0600 이 아닌 pgpass 를 무시한다 . Windows 마운트 파일은 0777 → 복사 후 chmod 600
2. 사용법 : localhost:5050 → 4 강실습 → lab (pgvector-ko) → Query Tool → 블록 선택 후 F5

기동과 확인

첫 빌드 1분 30초 (12 코어) ~ 5분 (4 코어)

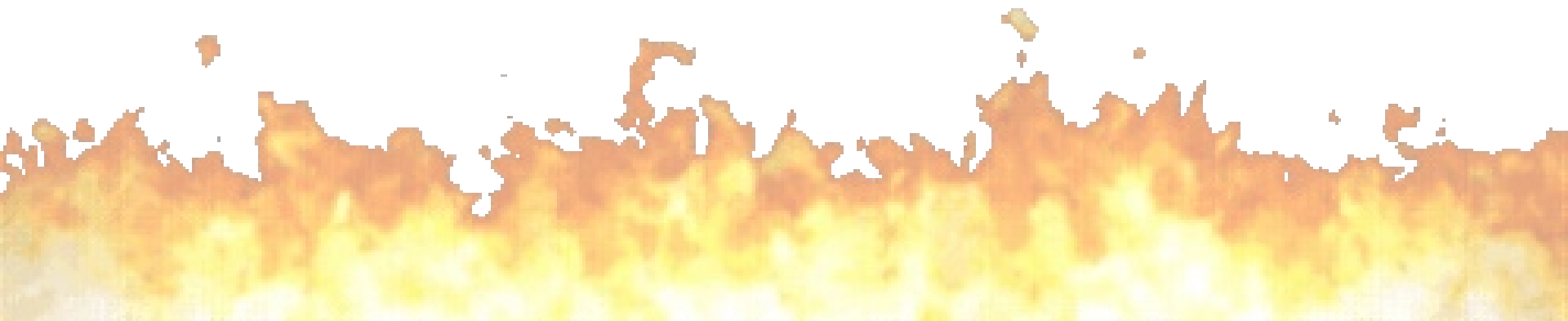
```
git clone https://github.com/hikaMaeng/-ai-.git
cd -ai-/4 강실습
docker compose up -d --build

docker compose ps
# ai4-db Up (healthy)
# ai4-pgadmin Up

docker compose exec db psql -U lab -d lab \
  -c "SELECT to_tsvector('korean', ' 무궁화꽃이 피었습니다 ')"
# ' 꽃 ':2 ' 무궁화 ':1 ' 피 ':3
```

조사 '이' 와 어미 '었습니다' 가 사라지고 , 위치 (1,2,3) 는 원문 순서를 기억한다

데이터 적재



질문마다 정답 지문이 있는 한국어 기계독해 데이터

필드	예시 (validation 첫 행)
title	BMW 코리아 , 창립 25 주년 기념 'BMW 코리아 25 주년 에디션 ' 한정 출시
context	BMW 코리아 (대표 한상윤) 는 창립 25 주년을 기념하는 ... 말라카이트 그린 다크 ... (약 1,000 자)
question	말라카이트에서 나온 색깔을 사용한 에디션은 ?
answer	뉴 740Li 25 주년 에디션
source	acrofan (뉴스)

- 1.validation 분할 : 질문 5,841 개 → 지문 중복 제거 후 5,309 개
- 한국경제 2,982 · 위키백과 1,623 · 아크로팬 704 (CC BY-SA 4.0)
2. 정답 지문이 있으므로 '상위 10 위 안에 찾았나'로 채점할 수 있다

로더 ① 다운로드와 정규화

datasets 라이브러리 없이 **parquet** 를 직접 받는다

```
def download(split):  
    url = f"https://huggingface.co/api/datasets/kclue/kclue/parquet/mrc/{split}/0.parquet"  
    path.write_bytes(requests.get(url).content)    # /cache 볼륨 → 재실행 시 생략  
    return pd.read_parquet(path)  
  
def build_frames(raw):  
    # 같은 지문에 질문이 여러 개 → 지문 내용의 md5 로 중복 제거  
    raw["doc_key"] = raw["context"].map(lambda s: hashlib.md5(s.encode()).hexdigest())  
    docs = raw.drop_duplicates("doc_key")[["doc_key", "title", "context", ...]]  
    qs    = raw[["guid", "doc_key", "question", ...]]  
    qs["answer"] = raw["answers"].map(lambda a: a["text"][0])  
    return docs, qs
```

허깅페이스는 공개 데이터셋을 parquet 로 자동 변환해 둔다 (/api/datasets/{id}/parquet/...)
무거운 datasets 패키지 없이 pandas 만으로 읽는다

로더 ② 텍스트 먼저 , 임베딩은 나중

실습을 막지 않도록 두 번 커밋한다

```
# 텍스트 : COPY → 커밋
cur.execute("DELETE FROM docs")      # TRUNCATE 는 행 트리거를 발동시키지 않는다
with cur.copy("COPY docs (doc_key, title, content, category, source) FROM STDIN") as cp:
    for d in docs.itertuples():
        cp.write_row((d.doc_key, d.title, d.content, d.category, d.source))
conn.commit()                        # ← 여기부터 ts_rank · BM25 실습 가능

# 임베딩 : 계산 → 임시 테이블 COPY → UPDATE 조인 → HNSW
model = TextEmbedding("sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2")
vecs = model.embed(texts, batch_size=64)      # 384 차원 , 입력 128 토큰
UPDATE docs x SET embedding = e.embedding FROM _emb e WHERE x.id = e.id;
CREATE INDEX docs_embedding_hnsw ON docs USING hnsw (embedding vector_cosine_ops);
```

1.tsv 는 생성 컬럼 → COPY 하는 동안 형태소 분석까지 끝난다 (5,309 건 15 초)

2.DELETE 를 쓰는 이유 : BM25 트리거 (5 장) 가 삭제를 감지해야 한다

로더 실행

'텍스트 적재 완료' 가 뜨면 창은 두고 다음 단계로

```
$ docker compose run --rm loader
```

1) 데이터셋 다운로드

validation: 5,841 질문

→ 지문 5,309 개 / 질문 5,841 개

2) 텍스트 적재 (COPY)

docs 5,309 행 / questions 5,841 행 , 15.0s

텍스트 적재 완료 (19s). 지금부터 10~24 번 실습을 시작해도 된다 .

3) 임베딩 - 25 번 실습에만 필요

docs 임베딩 5,309 건 114.3s

questions 임베딩 5,841 건 39.5s

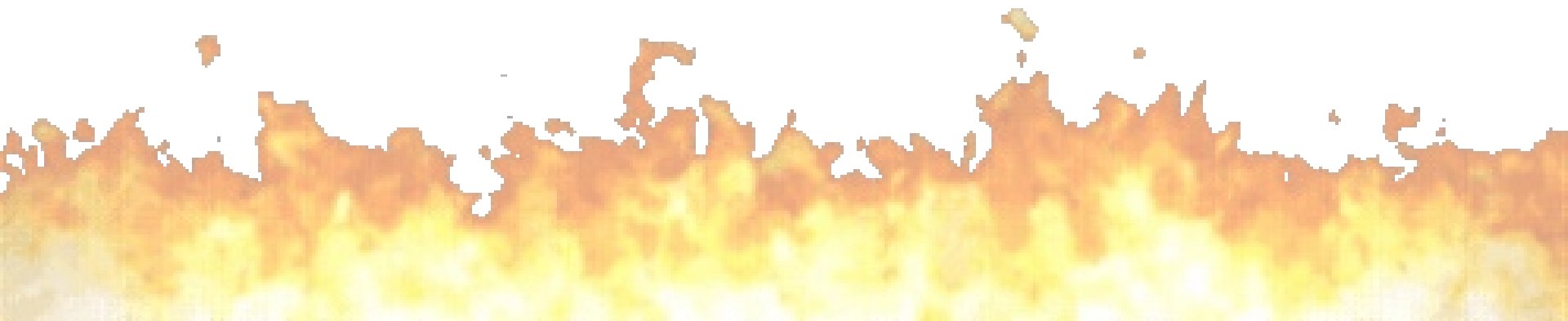
4) HNSW 인덱스 생성 (코사인 거리)

완료 (188s)

옵션 : --no-embed (텍스트만) · --embed-only (임베딩만) · --splits train validation (전체)

주의 : run --build 로 실행하면 db 컨테이너가 재생성되어 실행 중인 쿼리가 끊긴다

형태소 분석과 tsvector



mecab 은 어떻게 쪼개나

mecabko_analyze() : 형태소와 품사 태그

```
SELECT word, type FROM mecabko_analyze(' 아버지가방에들어가신다 ');
```

word	type
아버지	NNG 일반명사
가	JKS 주격조사
방	NNG 일반명사
에	JKB 부사격조사
들어가	VV 동사
시	EP 선어말어미
ㄴ다	EC 연결어미

mecabko_analyze(' 가셨습니다 ') ...(' 인공지능모델학습데이터 ')

word	type
갈	VV
시	EP
었	EP
습니다	EF

word	detail
인공지능	인공 + 지능
모델	
학습	
데이터	

띄어쓰기가 없어도 ' 아버지가 방에 ' 로 분리하고 복합명사도 쪼갬다
' 가셨습니다 → 갈 ' 처럼 원형 복원이 틀리기도 한다 → 사전 품질이 검색 품질의 상한

ts_debug : 토큰별 사전

```
SELECT alias, token, dictionaries, lexemes  
FROM ts_debug('korean', 'PostgreSQL 17에서 벡터 검색을 테스트했다');
```

alias	token	dictionaries	lexemes
asciiword	PostgreSQL	{english_stem}	{postgresql}
uint	17	{simple}	{17}
word	에서	{korean_stem}	{에서}
word	벡터	{korean_stem}	{벡터}
word	검색	{korean_stem}	{검색}
blank	을	{}	
word	테스트	{korean_stem}	{테스트}
word	했	{korean_stem}	{했}

1. 사전 배정

- 한글 → korean_stem
- 영문 → english_stem
- 숫자 → simple

2. 관찰

- '17에서' 의 '에서' 가 토큰으로 남음
- '했' 이 '하' 로 복원 안 됨

3. 불용어 · 원형 처리의 한계

- ES · Solr 는 불용어 사전으로 보완 (강의)

tsvector 와 setweight

형태소 → 위치 목록 + 위치별 가중치 (A~D)

```
SELECT * FROM unnest(to_tsvector('korean', ' 고양이 고양이 고양이 사료 '));
```

lexeme	positions	weights
고양이	{1,2,3}	{D,D,D}
사료	{4}	{D}

```
SELECT setweight(to_tsvector('korean', ' 고양이 사료 '), 'A')  
       || setweight(to_tsvector('korean', ' 강아지 간식과 고양이 장난감 '), 'D');  
-- ' 간식 ':4 ' 강아지 ':3 ' 고양이 ':1A,5 ' 사료 ':2A ' 장난감 ':6
```

1.positions 의 개수 = 문서 안 등장 횟수 (TF) → BM25 구현의 재료

2.setweight 는 위치마다 딱지를 붙인다

-|| 로 이르면 뒤쪽 위치가 자동으로 밀린다 (5)

- 딱지가 없으면 D (강의 : A 헤드라인 · B 부제 · C 본문 · D 주석)

함수	입력	tsquery
plainto_tsquery	고양이 사료를 추천해 주세요	' 고양이 ' & ' 사료 ' & ' 추천 '
websearch_to_tsquery	고양이 or 강아지 - 간식	' 고양이 ' ' 강아지 ' & ! ' 간식 '
to_tsquery	인공지능 <-> 모델	' 인공지능 ' <-> ' 모델 ' (인접)

```
EXPLAIN ANALYZE
```

```
SELECT id, title FROM docs WHERE tsv @@ plainto_tsquery('korean', ' 반도체 수출 ');
```

```
Bitmap Heap Scan on docs (actual rows=7)
```

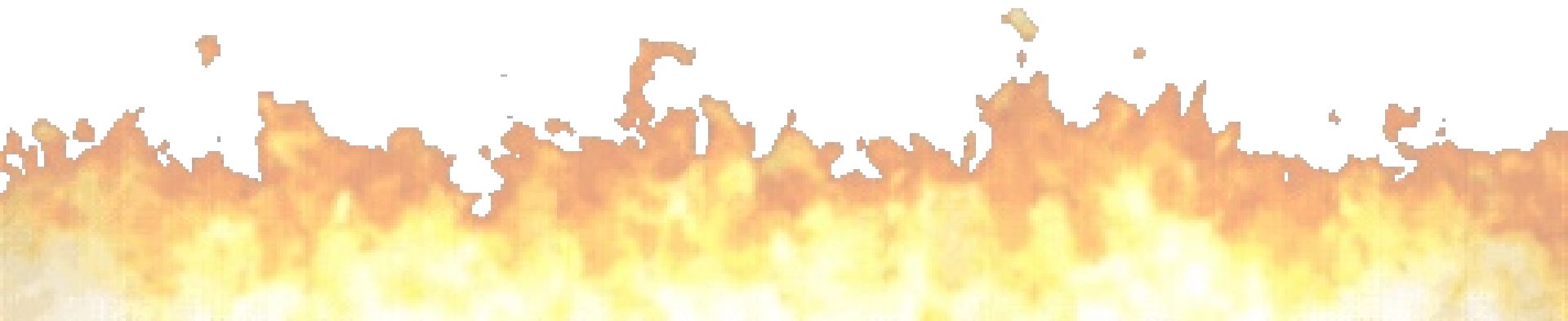
```
-> Bitmap Index Scan on docs_tsv_gin
```

```
Index Cond: (tsv @@ ' ' 반도체 ' ' & ' ' 수출 ' '::tsquery)
```

```
Execution Time: 0.427 ms
```

질의도 같은 형태소 분석을 거친다 → '사료를' 이 '사료' 로 바뀌어 매칭
@@ 는 GIN 역색인을 탄다 : 5,309 개 지문에서 0.4ms

ts_rank 계열의 동작



ts_rank 기본

@@ 로 거르고, ts_rank 로 정렬

```
SELECT title, ts_rank(tsv, q) AS rank
FROM docs, plainto_tsquery('korean', ' 반도체 수출 ') q
WHERE tsv @@ q
ORDER BY rank DESC LIMIT 10;
```

title	rank
...날개 단 SK 하이닉스 , 반도체 수출 14 조 최대	0.992
반도체 검사장비 개발 ... 세계 선두업체에 수출	0.909
中 " 美 마이크론 사겠다 " ... 韓 반도체 ' 초비상 '	0.176
한국 中企제품 , 中 알리바바서 판매...	0.097
" 日기술자 덕에 실패했던 부품개발 성공 "	0.000...

상위 두 건은 제목 (A) 에 ' 반도체 ' 와 ' 수출 ' 이 모두 있다 → 가중치 1.0

구역 가중치 배열

ts_rank('{D, C, B, A}', tsv, q) — D 부터 !

```
SELECT title,  
       ts_rank(tsv, q) AS 기본,           -- {0.1, 0.2, 0.4, 1.0}  
       ts_rank('{0.1, 0.2, 0.4, 0.1}', tsv, q) AS 제목가중치_꿈, -- A → 0.1  
       ts_rank('{1.0, 0.2, 0.4, 1.0}', tsv, q) AS 본문도_1점     -- D → 1.0  
FROM docs, plainto_tsquery('korean', ' 삼성전자 ') q WHERE tsv @@ q  
ORDER BY 기본 DESC LIMIT 10;
```

title	기본	제목 꿈	본문 1 점
고용부 " 삼성전자서비스 불법 파견 아니다 "...	0.644	0.097	0.967
분기 영업이익 3 조 ? ... 비상걸린 삼성전자...	0.643	0.096	0.963
삼성전자 '7 兆 영업이익' 회복 ...	0.640	0.093	0.929
삼성전자 또 불산 누출	0.636	0.089	0.890

제목에 있으면 0.6 점대 , 가중치를 끄면 0.1 아래 → 점수의 대부분이 ' 제목에 있느냐 ' 에서 나온다

반복 실험 : ts_rank 는 포화

같은 단어를 n 번 반복한 문서의 점수

```
SELECT n, ts_rank(t, q), ts_rank_cd(t, q)
FROM generate_series(1, 10) n,
     LATERAL (SELECT to_tsvector('korean', repeat(' 고양이 ', n) || ' 사료 추천 ') AS t,
                    plainto_tsquery('korean', ' 고양이 ') AS q) x;
```

n	1	2	3	4	5	6	8	10
ts_rank	.0608	.0760	.0827	.0865	.0890	.0907	.0929	.0942
ts_rank_cd	0.1	0.2	0.3	0.4	0.5	0.6	0.8	1.0

- 1.ts_rank 는 증가폭이 줄어들며 수렴한다 (포화)
 - 강의 슬라이드의 'saturation 이 없어 선형 ' 과 다르다
2. 오히려 ts_rank_cd 가 선형
 - 단어 하나짜리 커버가 n 개 → 커버 점수가 n 번 합산된다

포화의 정체 : tsrank.c

OR 질의 (calc_rank_or) 의 계산

```
/* 한 단어가 문서에 j 번째로 등장할 때마다 */
for (j = 0; j < dimt; j++)
    resj = resj + wpos(post[j]) / ((j + 1) * (j + 1));      /* w / (j+1)^2 */

/* limit (sum(1/i^2), i=1,inf) = pi^2/6 */
res = res + (wjm + resj - wjm / ((jm + 1) * (jm + 1))) / 1.64493406685;
```

등장	1 번째	2 번째	3 번째	4 번째	...	합의 극한
더해지는 값	w	w/4	w/9	w/16	...	$w \cdot \pi^2/6 \approx 1.645w$

1. 등장 횟수가 늘어도 기여가 $1/j^2$ 로 줄어 상한이 생긴다
2. BM25 의 $tf \cdot (k1+1)/(tf+k1)$ 와 모양은 다르지만 목적은 같다
3. ts_rank 에 없는 것은 포화가 아니라 IDF 와 평균 길이 (avgdl)

AND 의 함정

문장형 질문을 `plainto_tsquery` 로 넣으면

```
SELECT plainto_tsquery('korean', ' 말라카이트에서 나온 색깔을 사용한 에디션은 ?' );  
-- ' 말라카이트 ' & ' 나오 ' & ' 색깔 ' & ' 사용 ' & ' 에디션 '  
  
SELECT count(*) FROM docs  
WHERE tsv @@ plainto_tsquery('korean', ' 말라카이트에서 나온 색깔을 사용한 에디션은 ?' );  
-- 0
```

형태소 5 개를 모두 포함한 지문은 없다 → 0 건

정답 지문에는 ' 말라카이트 ' 와 ' 에디션 ' 만 있다 . 질문은 지문을 그대로 베끼지 않는다

RAG 의 질의는 대부분 문장이다 → OR 로 후보를 넓히고 , 순위 함수로 가려야 한다

OR 로 풀기

& 를 | 로 바꾸는 함수를 만들어 계속 쓴다

```
CREATE FUNCTION or_query(q text) RETURNS tsquery LANGUAGE sql IMMUTABLE AS $$  
    SELECT replace(plainto_tsquery('korean', q)::text, '&', '|')::tsquery  
$$;  
SELECT title, ts_rank(tsv, or_query(' 말라카이트에서 나온 색깔을 사용한 에디션은 ? ')) AS rank  
FROM docs WHERE tsv @@ or_query('...') ORDER BY rank DESC LIMIT 5;
```

#	title	rank
1	람보르기니 , 씨벨로와 협업한 리미티드 에디션 자전거	0.1555
2	BMW 코리아 25 주년 에디션 한정 출시 ← 정답	0.1448
3	1 년반 시범시행 했지만 사용률 12% 불과 ... 도로명 주소	0.1447
4	5 만弗 이상 사용자도 886 명	0.1406
5	창원 더시티세븐 호텔 경매 나왔다 ...	0.1389

' 말라카이트 ' 는 5,309 개 중 1 건에만 나오는 희귀어 . ts_rank 는 흔한 ' 사용 '.' 나오 ' 와 똑같이 센다

ts_rank_cd : 커버 밀집도

질의어를 모두 포함하는 최소 구간 (커버) 이 짧을수록 고점

```
WITH t(name, body) AS (VALUES
  ('A 붙어있음 ', ' 인공지능 모델 성능 평가 보고서 '),
  ('B 떨어짐 ', ' 인공지능 분야 최근 동향 정리 시장 규모 투자 현황 모델 비교 '),
  ('C 멀리 ', ' 인공지능 ' || repeat(' 기타 ', 30) || ' 모델 '))
SELECT name, ts_rank(to_tsvector('korean', body), q),
       ts_rank_cd(to_tsvector('korean', body), q)
FROM t, to_tsquery('korean', ' 인공지능 & 모델 ') q; -- 또는 ' 인공지능 | 모델 '
```

문서	AND ts_rank	AND ts_rank_cd	OR ts_rank	OR ts_rank_cd
A 붙어있음	0.0991	0.1	0.0608	0.2
B 8 단어 사이	0.0517	0.0111		
C 30 단어 사이	0.00004	0.0032	0.0608	0.2

1.AND : 둘 다 거리에 따라 감점 (ts_rank 도 calc_rank_and 에서 거리를 쓴다)

2.OR : 단어 하나로 커버가 성립 → 거리 효과가 사라진다. 커버 여러 개는 합산 (1 개 0.1 · 2 개 0.225 · 3 개 0.4)

길이 정규화 옵션

`ts_rank(tsv, q, 옵션)` — 문서 길이로 나눈다

옵션	나누는 값	성격
0	없음 (기본)	
1	$1 + \log(\text{문서 길이})$	완만한 감쇄 — 긴 문서도 커버리지 보존
2	문서 길이	급격한 감쇄 — 짧은 문서가 유리
8 / 16	고유 단어 수 / $1 + \log(\text{고유 단어 수})$	
32	$\text{rank} / (\text{rank} + 1)$	0~1 로 눌러 담기, 조합 가능 (1 32)

' 서울 아파트 가격 '	opt0	opt1	opt2
짧음 (5 단어)	0.0608	0.0235	0.0122
중간 (40 단어)	0.0608	0.0113	0.0015
김 (300 단어)	0.0608	0.0074	0.0002

- 실제 질의 상위 10 개 평균 길이
- opt0 1,279 자 → opt1·2 825 자
- 한계
- 코퍼스 평균 길이를 모른다 → BM25 의 b 불가

품질 측정 : Hit@10 · MRR

질문 500 개로 '정답 지문을 몇 위에 찾았나'

```
CREATE TABLE eval_q AS
SELECT id, doc_id, question FROM questions WHERE id % 11 = 0 LIMIT 500;

-- 질문마다 상위 10 개를 뽑아 정답 (doc_id) 의 순위를 본다
FROM eval_q e
CROSS JOIN LATERAL (SELECT or_query(e.question) AS q) qq -- 질문당 1 번만 분석
CROSS JOIN LATERAL (
    SELECT d.id, row_number() OVER (ORDER BY ts_rank(d.tsv, qq.q) DESC) AS rn
    FROM docs d WHERE d.tsv @@ qq.q ORDER BY rn LIMIT 10) r
```

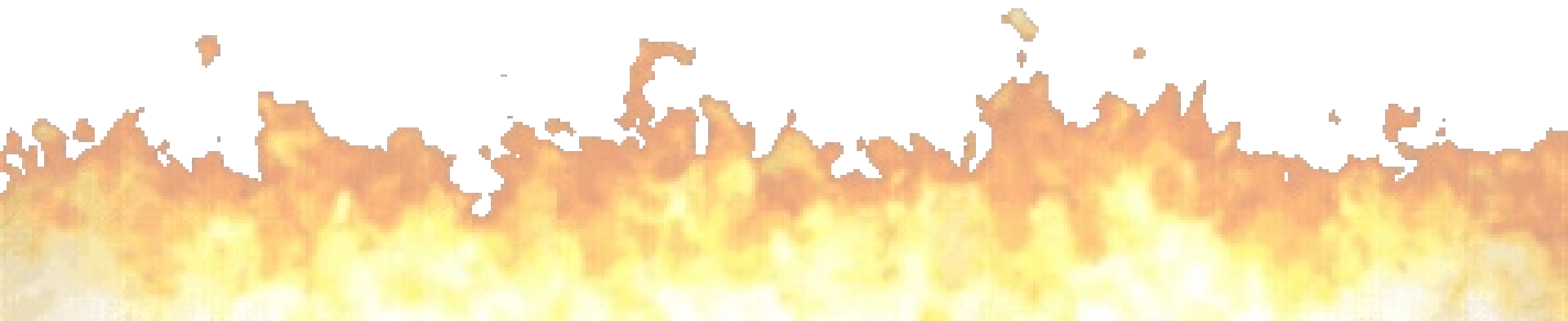
1. Hit@10 = 정답이 10 위 안에 든 질문의 비율 · MRR = 정답 순위 역수의 평균 (1 위 1.0 · 2 위 0.5 · 밖 0)

2. 성능 함정 : tsquery 를 문서 행마다 만들면 $500 \times 5,309$ 번 형태소 분석 → 4 분 . 질문당 1 번으로 바뀌 2 분

방법	Hit@10	MRR	해석
AND + ts_rank	0.048	0.044	정확하지만 거의 못 찾음 (재현율 붕괴)
OR + ts_rank	0.484	0.354	흔한 단어가 많은 문서에 끌려감
OR + ts_rank_cd	0.524	0.329	Hit 는 조금 높고 순위는 낮음
OR + ts_rank 정규화 1	0.484	0.354	198 개 질문에서 순위 변동 , 합계는 같음
OR + ts_rank 정규화 2	0.374	0.233	짧은 지문을 과하게 끌어올림

1. 절반의 질문은 정답을 10 위 안에서도 못 찾는다
2. 지문 길이가 고른 코퍼스 (형태소 31~358 개) 에서는 정규화의 효과가 작다
3. 무엇이 빠졌나 ? → 단어의 희귀도 (IDF) 와 코퍼스 평균 길이

BM25 직접 구현



$$\text{BM25}(D, Q) = \sum_{t \in Q} \text{IDF}(t) \cdot f(t, D) \cdot (k_1 + 1) / (f(t, D) + k_1 \cdot (1 - b + b \cdot |D| / \text{avgdl}))$$
$$\text{IDF}(t) = \ln(1 + (N - \text{df}(t) + 0.5) / (\text{df}(t) + 0.5)) \quad \text{-- Lucene 변형}$$

기호	뜻	어디서 구하나
$f(t, D)$	문서 D 안에서 단어 t 의 등장 횟수 (TF)	tsvector 의 positions 개수
$ D $	문서 길이 (의미 형태소 수)	문서의 TF 합
avgdl	코퍼스 평균 문서 길이	전체 길이 합 / N
$N \cdot \text{df}(t)$	전체 문서 수 $\cdot t$ 를 포함한 문서 수	집계
$k_1 \cdot b$	TF 포화 강도 (1.2) $\cdot$ 길이 보간 강도 (0.75)	파라미터

형태소 분석은 이미 tsvector 에 끝나 있다 → 남은 일은 ' 세고 , 저장하고 , 점수 매기기 '

공식을 테이블로

질의어로 찾아 들어가는 역색인 + 통계

```
CREATE TABLE bm25_tf (                                -- 역색인 : f(t,D)
    term text, doc_id bigint, tf int,
    PRIMARY KEY (term, doc_id));                        -- term 이 앞 → 질의어로 바로 탐색
CREATE INDEX bm25_tf_doc ON bm25_tf(doc_id); -- 문서 수정 · 삭제용

CREATE TABLE bm25_doclen (doc_id bigint PRIMARY KEY, len int); -- |D|
CREATE TABLE bm25_df      (term text PRIMARY KEY, df int);      -- df(t)
CREATE TABLE bm25_idf      (term text PRIMARY KEY, idf float8);  -- IDF 캐시
CREATE TABLE bm25_stats    (id int PRIMARY KEY DEFAULT 1 CHECK (id = 1),
                             n_docs bigint, total_len bigint);    -- N, avgdl
```

1. PK 순서가 곧 설계 : (term, doc_id) = ' 이 단어가 나온 문서들 '

2. bm25_stats 는 CHECK (id = 1) 로 항상 1 행 · IDF 는 N 이 바뀌면 전부 바뀌므로 캐시로 분리

```
SELECT t.lexeme AS term, cardinality(t.positions) AS tf, t.weights  
FROM docs d, unnest(d.tsv) t  
WHERE d.title LIKE 'BMW 코리아 , 창립 25주년%' ORDER BY tf DESC LIMIT 6;
```

term	tf	weights
25	14	{A,A,D,D,D,D,D,D,D,D,D,D,D,D,D}
주년	12	{A,A,D,D,D,D,D,D,D,D,D,D,D}
에디션	10	{A,D,D,D,D,D,D,D,D,D,D}
한정	8	{A,D,D,D,D,D,D,D,D}
bmw	8	{A,A,D,D,D,D,D,D,D}
시리즈	8	{D,D,D,D,D,D,D,D,D}

1.unnest(tsv)

-lexeme → term

-cardinality(positions) → tf

2. 주의

- 형태소당 위치는 최대 256 개까지만 저장

- 이 코퍼스의 최대 tf 는 49 → 문제없음

```
INSERT INTO bm25_tf (term, doc_id, tf) -- ① TF
SELECT t.lexeme, d.id, cardinality(t.positions) FROM docs d, unnest(d.tsv) t;

INSERT INTO bm25_doclen (doc_id, len) -- ② 문서 길이
SELECT d.id, (SELECT sum(cardinality(t.positions)) FROM unnest(d.tsv) t) FROM docs d;

UPDATE bm25_stats SET n_docs = (SELECT count(*) FROM bm25_doclen), -- ③ N, 전체 길이
total_len = (SELECT sum(len) FROM bm25_doclen);

INSERT INTO bm25_df SELECT term, count(*) FROM bm25_tf GROUP BY term; -- ④ DF

CREATE FUNCTION bm25_refresh_idf() RETURNS void LANGUAGE sql AS $$ -- ⑤ IDF
    TRUNCATE bm25_idf;
    INSERT INTO bm25_idf SELECT d.term, ln(1 + (s.n_docs - d.df + 0.5) / (d.df + 0.5))
    FROM bm25_df d, bm25_stats s;
$$;
```

결과 : 역색인 908,436 행 · 고유 단어 64,877 개 · N = 5,309 · avgdl = 281.3 · 약 30 초

IDF 가 말해 주는 것

흔한 단어	df	IDF
것	4,215	0.231
있	4,093	0.260
들	4,024	0.277
등	3,950	0.296
하	3,877	0.314
년	3,598	0.389

희귀 단어	df	IDF
말라카이트	1	8.172
트랜스포머	2	7.661
에디션	16	5.774
반도체	71	4.308
사용	906	1.768

df 구간	단어 수
1	31,860
2~5	18,941
6~50	11,225
51~500	2,585
500+	266

1. 강의의 '은 · 는 · 이 · 가 · 이다는 거의 0 점' : 것 · 있 · 들 · 하의 IDF 는 0.2~0.3
2. '말라카이트' 한 단어가 '것' 35 개와 맞먹는다
3. 단어의 절반 (49%) 이 문서 1 개에만 나온다 — 지프의 법칙

점수를 항 단위로 쪼개기

'말라카이트에서 나온 색깔을 사용한 에디션은 ?' × 정답 지문

term	df	IDF	tf	D	avgdl	tf 항	점수
말라카이트	1	8.172	3	269	281.3	1.586	12.963
에디션	16	5.774	10	269	281.3	1.971	11.382
색깔	31	5.127	0			0	0
사용	906	1.768	0			0	0
나오	1,101	1.573	0			0	0

- 총점 24.35 는 희귀어 두 개가 만든다
- tf 3 인 '말라카이트' 가 tf 10 인 '에디션' 보다 높다
 -IDF 8.2 × 포화된 tf 항 1.59 vs IDF 5.8 × 1.97
- 질문의 나머지 단어는 지문에 없다 — 사람의 질문은 지문을 베끼지 않는다

k1 : TF 포화 곡선

$|D| = \text{avgdl}$ 일 때 tf 항 = $\text{tf} \cdot (k1+1) / (\text{tf} + k1)$, 상한 $k1+1$

tf	1	2	3	5	10	20	50
k1 = 1.2	1.00	1.38	1.57	1.77	1.96	2.08	2.15
k1 = 2.0	1.00	1.50	1.80	2.14	2.50	2.73	2.89
k1 = 5.0	1.00	1.71	2.25	3.00	4.00	4.80	5.45
포화 없음	1	2	3	5	10	20	50

1.k1 이 클수록 포화가 늦다 = TF 가 점수에 오래 기여한다

- 강의 슬라이드의 '클수록 TF 억제' 는 반대

2.k1 = 1.2 는 tf 5 에서 이미 상한 (2.2) 의 80%

- '고양이 x4' 문서가 '고양이 사료 추천' 을 이기지 못하는 이유

b : 길이 보간

tf = 3 고정, 문서 길이가 평균의 몇 배인가

D / avgdl	0.25	0.5	1.0	2.0	4.0
b = 0	1.571	1.571	1.571	1.571	1.571
b = 0.75	1.872	1.760	1.571	1.294	0.957
b = 1	2.000	1.833	1.571	1.222	0.846

1. 평균 길이에서는 b 와 무관 · 평균보다 길면 감점, 짧으면 가점

2. PG 정규화와의 차이

-PG : '이 문서의 길이' 로만 나눈다

-BM25 : '평균 대비' 로 보간한다 → avgdl 이 필요한 이유

bm25_search() 함수



```
CREATE FUNCTION bm25_search(q text, k int DEFAULT 10,  
                           k1 float8 DEFAULT 1.2, b float8 DEFAULT 0.75)  
RETURNS TABLE (doc_id bigint, score float8) LANGUAGE sql STABLE AS $$  
  WITH qt AS (SELECT DISTINCT lexeme AS term                                -- 질의도 형태소 분석  
               FROM unnest(to_tsvector('korean', q))),  
       s AS (SELECT total_len::float8 / n_docs AS avgd1 FROM bm25_stats)  
  SELECT t.doc_id,  
         sum(i.idf * t.tf * (k1 + 1)  
           / (t.tf + k1 * (1 - b + b * dl.len / s.avgd1))) AS score  
  FROM qt  
  JOIN bm25_tf      t  ON t.term = qt.term                                -- 역색인 PK 탐색  
  JOIN bm25_idf     i  ON i.term = qt.term  
  JOIN bm25_doclen  dl ON dl.doc_id = t.doc_id  
  CROSS JOIN s  
  GROUP BY t.doc_id ORDER BY score DESC LIMIT k  
$$;
```

공식이 SQL 한 문장 · 질의어가 있는 행만 읽는다 (bm25_tf_pkey 인덱스 스캔, 약 35ms) · k1 · b 는 인자로

같은 질문 , 다른 순위

'말라카이트에서 나온 색깔을 사용한 에디션은 ?'

OR + ts_rank

#	title	rank
1	람보르기니 리미티드 에디션 자전거	0.156
2	BMW 코리아 25 주년 에디션 ← 정답	0.145
3	도로명 주소 사용률 12% 불과	0.145
4	5 만弗 이상 사용자도 886 명	0.141
5	창원 호텔 경매 나왔다	0.139

BM25

#	title	score
1	BMW 코리아 25 주년 에디션 ← 정답	24.35
2	유튜브스 라이프 OMG 에디션	12.21
3	MINI 새빌 로 에디션 출시	11.55
4	람보르기니 리미티드 에디션 자전거	11.07
5	조흔색 (광물의 색깔)	10.01

ts_rank 에서는 ' 사용 '.' 나오 ' 가 들어간 무관한 기사가 섞였다

BM25 에서는 희귀어 (에디션 · 색깔) 를 가진 문서만 올라오고 , 정답이 2 배 차이로 1 위

트리거 설계

문서가 바뀌면 역색인도 따라 바뀌어야 한다

1. 문장 (statement) 단위 트리거 + 전이 테이블

- 행 단위면 1 만 건 INSERT 에 트리거 1 만 번

- REFERENCING NEW TABLE AS new_rows → 문장당 1 번 , 바뀐 행 전체를 집합으로

- PG 제약 : 전이 테이블은 이벤트 하나짜리 트리거에만 → INSERT · UPDATE · DELETE 3 개

2.UPDATE = 옛 문서 빼기 + 새 문서 더하기

- tsv 가 그대로인 UPDATE (임베딩만 채우기 등) 는 건너뛴다

3.IDF 는 ' 건드린 단어 ' 만 즉시 갱신

- N 이 바뀌면 모든 단어의 IDF 가 조금씩 바뀐다 → 매번 전부 고치면 너무 비싸다

- 부분 갱신 + 주기적 bm25_refresh_idf() — Lucene 세그먼트 병합과 같은 발상

4.bm25_rebuild() : 트리거가 꺼져 있던 동안의 불일치를 복구하는 전체 재구축

```
CREATE FUNCTION bm25_sync() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    -- 전이 테이블은 이 함수 안에서만 보인다 → 임시 테이블로 옮겨 하위 함수와 공유
    IF to_regclass('pg_temp._bm25_old') IS NULL THEN
        CREATE TEMP TABLE _bm25_old (id bigint, tsv tsvector);
        CREATE TEMP TABLE _bm25_new (id bigint, tsv tsvector);
    END IF;
    TRUNCATE _bm25_old, _bm25_new;

    IF TG_OP = 'INSERT' THEN INSERT INTO _bm25_new SELECT id, tsv FROM new_rows;
    ELSIF TG_OP = 'DELETE' THEN INSERT INTO _bm25_old SELECT id, tsv FROM old_rows;
    ELSIF TG_OP = 'UPDATE' THEN
        -- tsv 가 바뀐 행만
        INSERT INTO _bm25_old SELECT o.id, o.tsv FROM old_rows o JOIN new_rows n USING (id)
            WHERE o.tsv IS DISTINCT FROM n.tsv;
        INSERT INTO _bm25_new SELECT n.id, n.tsv FROM old_rows o JOIN new_rows n USING (id)
            WHERE o.tsv IS DISTINCT FROM n.tsv;
    END IF;
    PERFORM bm25_remove_docs(); PERFORM bm25_add_docs(); PERFORM bm25_touch_idf();
    RETURN NULL;
END $$;
```

```
-- bm25_add_docs() : df 는 ON CONFLICT 로 누적
INSERT INTO bm25_tf (term, doc_id, tf)
SELECT t.lexeme, n.id, cardinality(t.positions) FROM _bm25_new n, unnest(n.tsv) t;
INSERT INTO bm25_df (term, df)
SELECT t.lexeme, count(*) FROM _bm25_new n, unnest(n.tsv) t GROUP BY t.lexeme
ON CONFLICT (term) DO UPDATE SET df = bm25_df.df + EXCLUDED.df;

-- bm25_remove_docs() : 지운 tf 행을 RETURNING 으로 받아 df 감소
WITH del AS (DELETE FROM bm25_tf t USING _bm25_old o
              WHERE t.doc_id = o.id RETURNING t.term)
UPDATE bm25_df d SET df = d.df - c.n
FROM (SELECT term, count(*) n FROM del GROUP BY term) c WHERE d.term = c.term;

-- 트리거 3 개 : 전이 테이블은 이벤트 하나짜리 트리거에만
CREATE TRIGGER bm25_sync_ins AFTER INSERT ON docs
  REFERENCING NEW TABLE AS new_rows FOR EACH STATEMENT EXECUTE FUNCTION bm25_sync();
CREATE TRIGGER bm25_sync_upd AFTER UPDATE ON docs
  REFERENCING OLD TABLE AS old_rows NEW TABLE AS new_rows FOR EACH STATEMENT ...;
CREATE TRIGGER bm25_sync_del AFTER DELETE ON docs
  REFERENCING OLD TABLE AS old_rows FOR EACH STATEMENT ...;
```

검증 ① INSERT

```
INSERT INTO docs (doc_key, title, content, source) VALUES
('test-1', '양자컴퓨터 상용화 경쟁', '... 큐비트 100 개 규모의 양자컴퓨터 시제품 ... 큐비트 오류율 ...', 'test'),
('test-2', '말라카이트 원석 전시회', '녹색 광물 말라카이트 원석을 모은 전시회가 열린다.', 'test');
```

확인	전	후
N · total_len	5,309 · 1,493,622	5,311 · 1,493,652
큐비트 df · IDF	(없음)	1 · 8.172
말라카이트 df · IDF	1 · 8.172	2 · 7.661

bm25_search(' 큐비트 양자컴퓨터 ')	score
양자컴퓨터 상용화 경쟁 ← 방금 넣은 문서	32.26
정부 " 日 수산물 수입금지 ... "	9.66

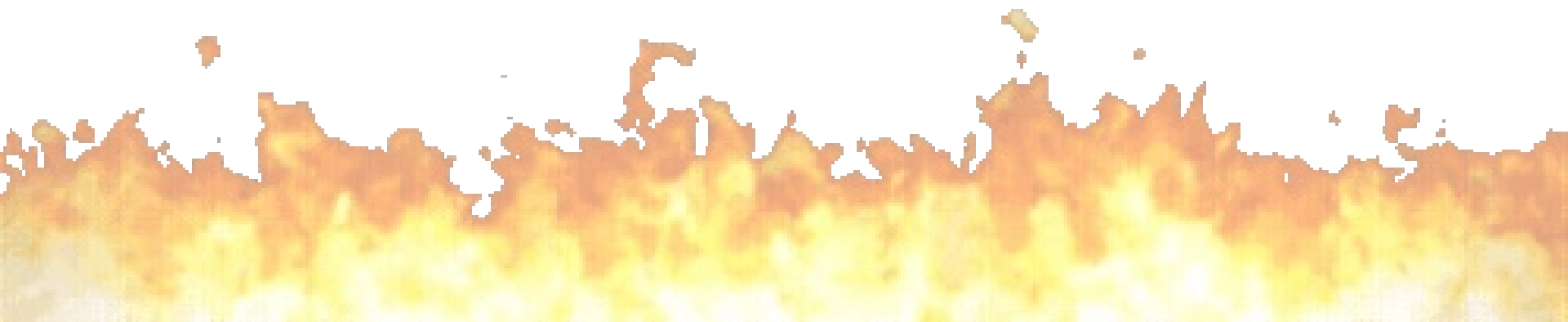
1. 넣자마자 검색된다
2. 희귀어가 하나 더 생기면 IDF 가 떨어진다
 - 말라카이트 8.17 → 7.66
 - 기존 정답 점수 24.35 → 23.53

동작	기대	결과
본문에서 '큐비트' 삭제 (UPDATE)	큐비트가 사전에서 사라짐	bm25_df 0 건 ✓
source 컬럼만 UPDATE	역색인 변화 없음	N · total_len 그대로 ✓
테스트 문서 DELETE	원래 통계로 복귀	5,309 · 1,493,622 ✓

-- 트리거로 유지한 값 vs 처음부터 다시 계산한 값 (EXCEPT 양방향 비교)
tf 불일치 0 · df 불일치 0 · N 불일치 0 · IDF 오차 (>1e-9) 7

IDF 오차 7 건은 설계대로다. UPDATE 때 빠진 단어 ('연구진' '오류율' 등) 는 그때의 N = 5,311 로 계산됐고, DELETE 는 수정된 본문의 단어만 건드리므로 다시 계산되지 않았다 → SELECT bm25_refresh_idf(); 후 0 건 로더 재실행 (DELETE 5,309 행 → COPY 5,309 행) 도 트리거 2 번으로 처리된다

종합 비교와 해석



```
-- 벡터 : 질문 임베딩과 지문 임베딩의 코사인 거리 (HNSW 인덱스 )
SELECT d.id FROM docs d ORDER BY d.embedding <=> q.embedding LIMIT 10;

-- RRF : 점수 대신 ' 순위 ' 만 합친다    score =  $\sum 1 / (60 + \text{rank})$ 
SELECT doc_id, sum(1.0 / (60 + rn)) AS rrf FROM (
  SELECT doc_id, row_number() OVER (ORDER BY score DESC) AS rn
  FROM bm25_search(question, 50)
  UNION ALL
  SELECT id, row_number() OVER (ORDER BY embedding <=> q.embedding)
  FROM (SELECT id, embedding FROM docs ORDER BY embedding <=> q.embedding LIMIT 50) v
) u GROUP BY doc_id ORDER BY rrf DESC LIMIT 10;
```

BM25 점수 (0~30) 와 코사인 거리 (0~2) 는 척도가 달라 더할 수 없다 → 순위만 쓰는 RRF

KLUE-MRC 질문 500 개, 지문 5,309 개

방법	Hit@10	MRR	무엇이 다른가
BM25 (직접 구현)	0.938	0.838	IDF + 평균 길이 보간
하이브리드 RRF	0.896	0.523	BM25 50 위 + 벡터 50 위 순위 합
OR + ts_rank_cd	0.524	0.329	밀집도
OR + ts_rank	0.484	0.354	PG 기본
벡터 (MiniLM, 지문 통째)	0.376	0.204	HNSW 코사인
AND + ts_rank	0.048	0.044	모든 단어 포함

같은 tsvector, 같은 형태소 분석 . 차이는 IDF 와 avgdl 뿐 → Hit@10 0.48 에서 0.94 로

파라미터와 실패 사례

k1	b	Hit@10
0.5	0.75	0.948
1.2	0	0.948
1.2	0.75	0.938
1.2	1.0	0.936
2.0	0.75	0.930

BM25 만 맞힌 질문	벡터만 맞힌 질문
슈트레제만이 이끌었던 당은 ?	실존하는 발광체로 만들 수 있는 색깔을 뜻하는 단어는 ?
2012 년 RIR 의 중위값은 ?	청와대가 독자적으로 내세운 정책의 명칭은 ?
코페르니쿠스가 처음 학위를 받은 학교는 ?	일본에서 가장 긴 고속도로의 마지막 지점 도시는 ?
법무법인 한결이 10 년차에 합병된 업체는 ?	인천시에 중국 국적 대학생이 가장 많았던 해는 ?

1. $b = 0$ 이 기본값과 같거나 낮다 → 지문 길이가 고른 코퍼스에서는 길이 보간이 잡음
2. BM25 는 고유명사 · 숫자에 강하고 , 벡터는 바뀌 말한 질문에 강하다

1. 데이터의 성격

- KLUE-MRC 질문은 지문의 고유명사 · 숫자를 그대로 쓰는 경우가 많다 → 키워드 검색에 유리

2. 벡터가 약한 이유 = 청킹

- MiniLM 의 입력은 128 토큰 → 1,000 자 지문의 앞 2~3 문장만 벡터에 담긴다
- 정답이 지문 뒤쪽에 있으면 벡터로는 찾을 수 없다 (강의 : ' 청킹이 가장 중요 '

3.RRF 가 BM25 보다 낮은 이유

- 두 목록을 동등하게 섞는다 → 약한 쪽이 강한 쪽의 순위를 끌어내린다
- 그래도 둘이 맞히는 질문이 다르다 → 가중치와 청킹을 고치면 역전 가능

4. 강의의 결론과 연결

- 검색 품질의 8 할은 모델이 아니라 코퍼스 정제 · 청킹 · 균형

1. 청킹 후 재측정

- 지문을 문장 2~3 개 단위로 나눠 chunks 테이블에 임베딩 → 벡터 Hit@10 이 얼마나 오르나

2. 가중 RRF

- BM25 쪽 순위에 가중치 2 → 하이브리드가 BM25 를 넘는가

3.BM25F

- 제목 (A) 과 본문 (D) 의 tf 를 따로 세어 가중 합산 — tsvector 의 weights 활용

4. 질의어 빈도 (qtf)

- 지금은 DISTINCT 로 질의어를 한 번씩만 센다 . 반복된 질의어에 가중을 주면 ?

5. 전체 데이터

- loader --splits train validation (지문 약 1.9 만)
- N 이 커지면 IDF 분포와 ts_rank · BM25 격차가 어떻게 변하나

```
docker compose down          # 컨테이너 정지 ( 데이터 유지 )
docker compose down -v      # 데이터 · 캐시까지 삭제 → 처음부터
```

```
# 포트 충돌 ( 5432 / 5050 사용 중 )
```

```
DB_PORT=15432 PGADMIN_PORT=15050 docker compose up -d          # macOS / Linux
$env:DB_PORT=15432; $env:PGADMIN_PORT=15050; docker compose up -d # PowerShell
```

- 1.pgAdmin 이 안 열림 → 첫 기동은 20~30 초 , docker compose logs pgadmin
- 2.25 번 벡터 결과가 0 → 임베딩 진행 중 . loader 의 ' 완료 ' 확인 또는 --embed-only
3. 실습 재료 전체 : github.com/hikaMaeng/-ai- / 4 강실습